CPU Scheduling Comparison Project (RR vs SJF)

Introduction

This project compares two CPU scheduling algorithms: Round Robin (RR) and Non-Preemptive Shortest Job First (SJF).

The purpose of the project is to analyze how each algorithm performs in terms of:

.Waiting Time (WT)

.Turnaround Time (TAT)

Response Time (RT)

.Fairness and CPU utilization

The project also studies the trade-off between fairness and efficiency in CPU scheduling.

Objectives

.Compare Round Robin and Non-Preemptive SJF scheduling algorithms

.Measure performance using WT, TAT, and RT

.Study fairness versus efficiency trade-offs

.Analyze the effect of Time Quantum in Round Robin

.Evaluate which algorithm performs better under different workloads

---

Algorithms Explanation

Round Robin (RR)

Round Robin is a preemptive time-sharing CPU scheduling algorithm where each process is assigned a fixed time quantum.

Processes are executed in a circular ready queue.
If a process does not finish during its quantum, it is moved to the end of the queue and the CPU is given to the next process.

This approach improves fairness and responsiveness because every process gets CPU time regularly.

Non-Preemptive SJF (Shortest Job First)

Non-Preemptive SJF selects the process with the smallest burst time among the arrived processes.

Once a process starts execution, it continues running until completion without interruption, even if a shorter process arrives later.

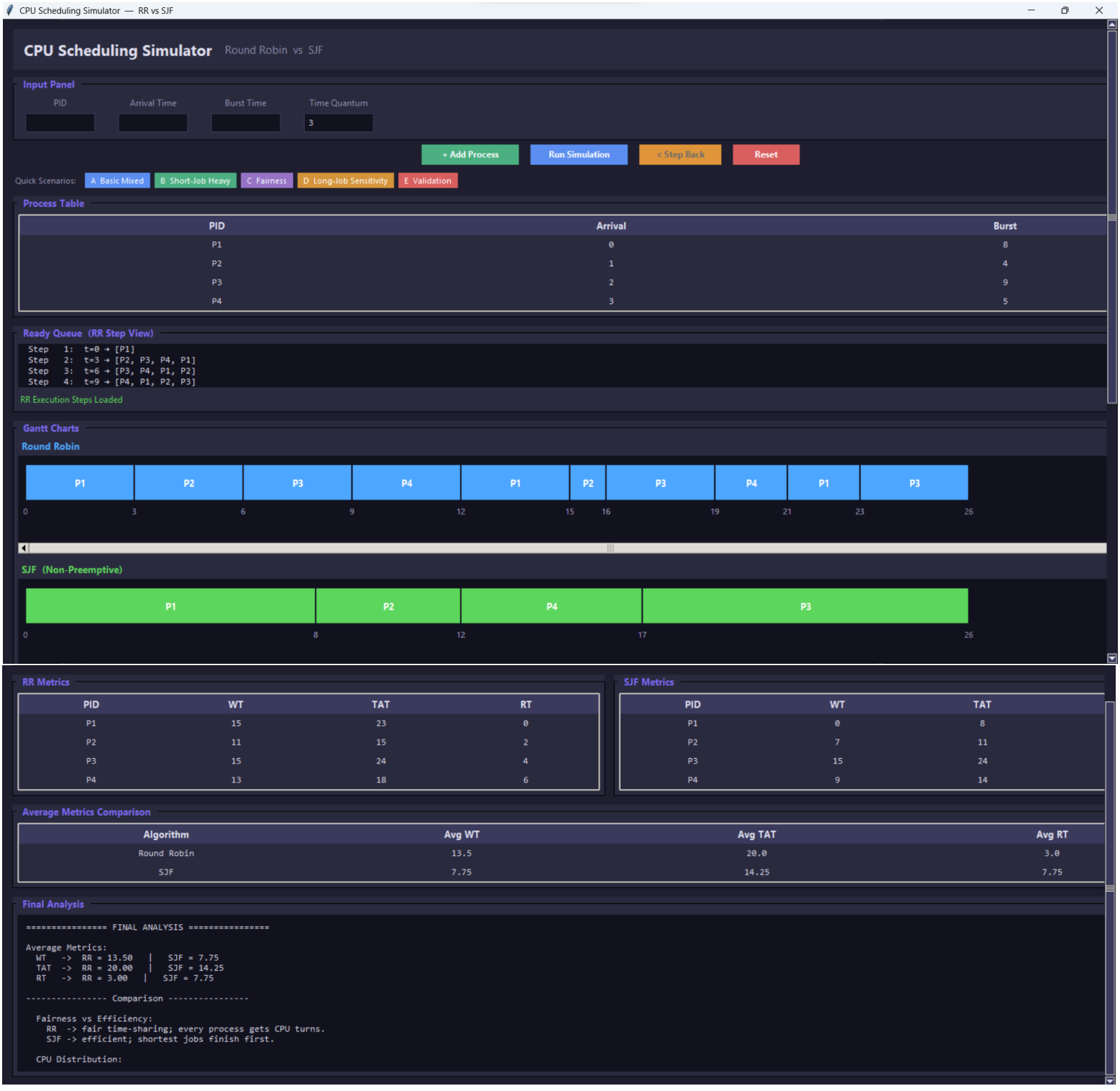
This algorithm is efficient in minimizing average waiting time and turnaround time, but long processes may experience longer waiting times.

Test Scenarios

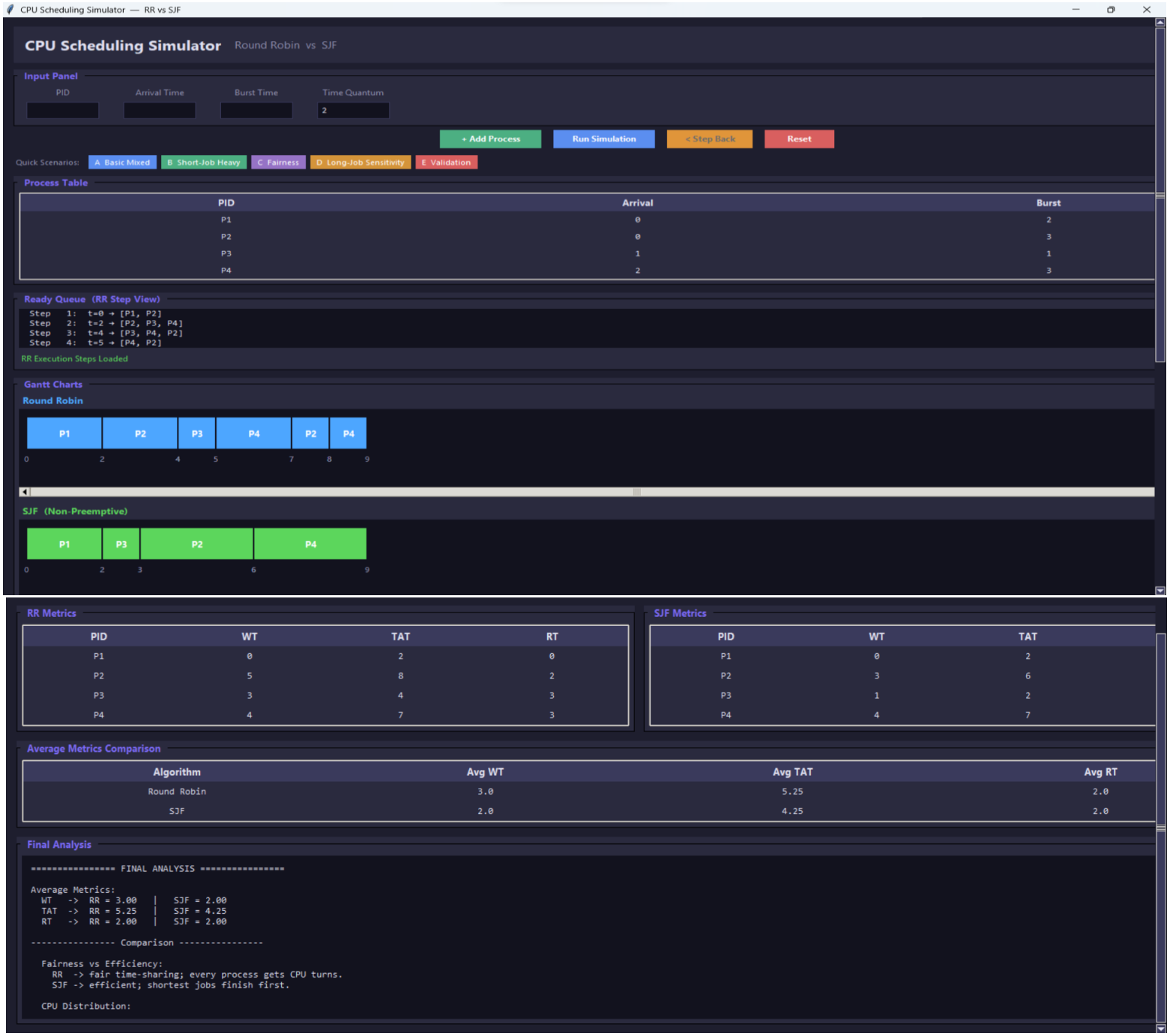
- A. Basic Mixed Workload
- B. Short Job Heavy Case
- C. Fairness Case
- D. Long Job Sensitivity Case
- E. Validation Case

Screenshots Section (Placeholders)

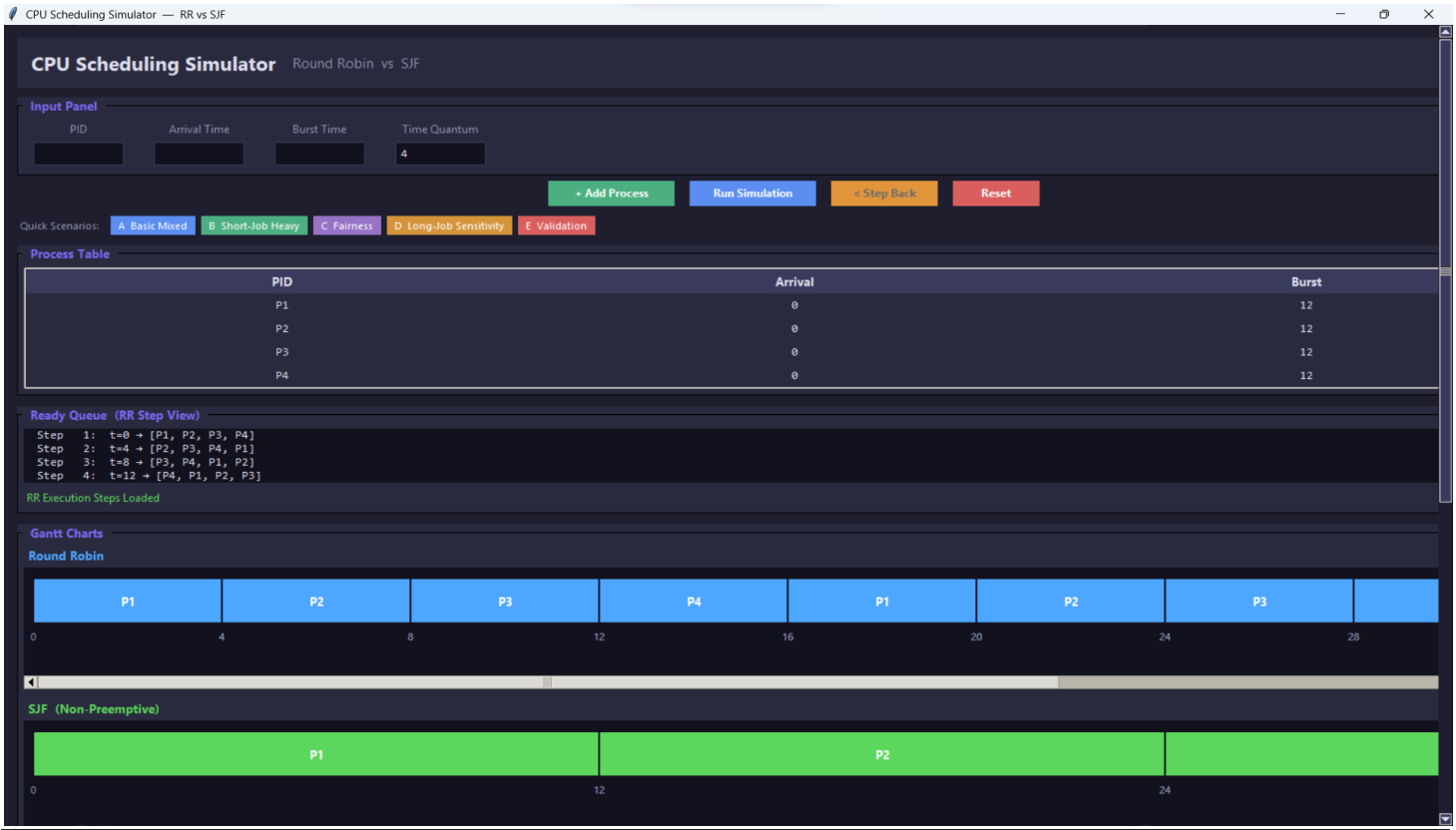
A. Basic Mixed Workload Screenshot Placeholder

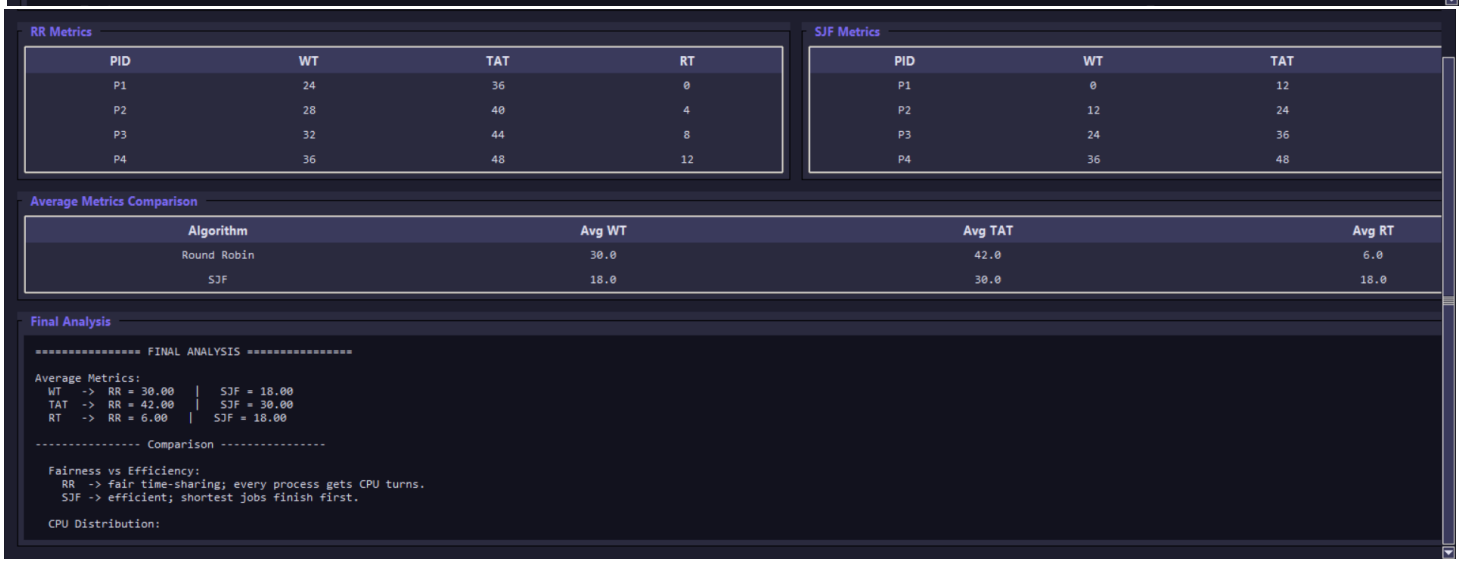


B. Short Job Heavy Case Screenshot Placeholder

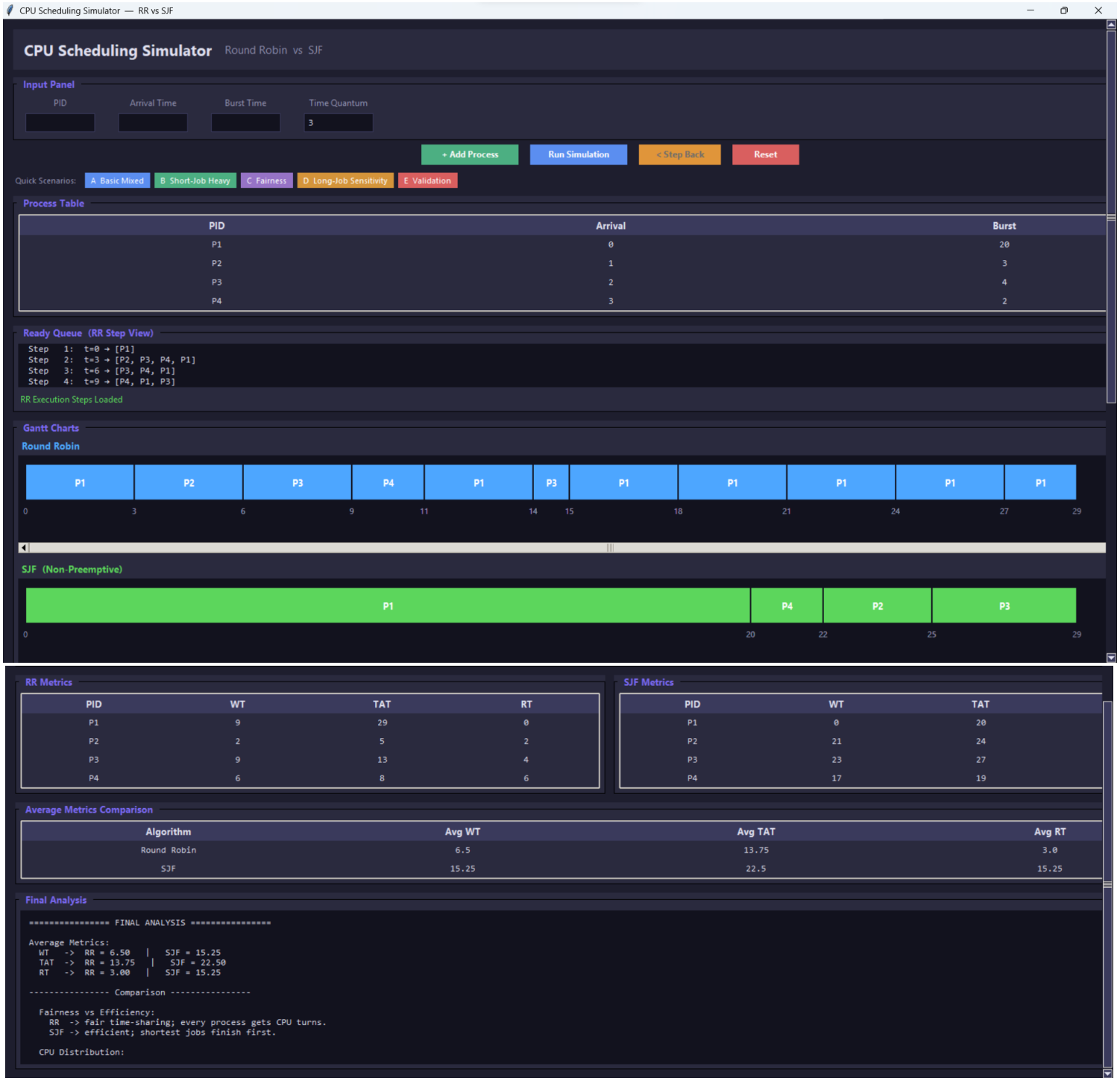


C. Fairness Case Screenshot Placeholder

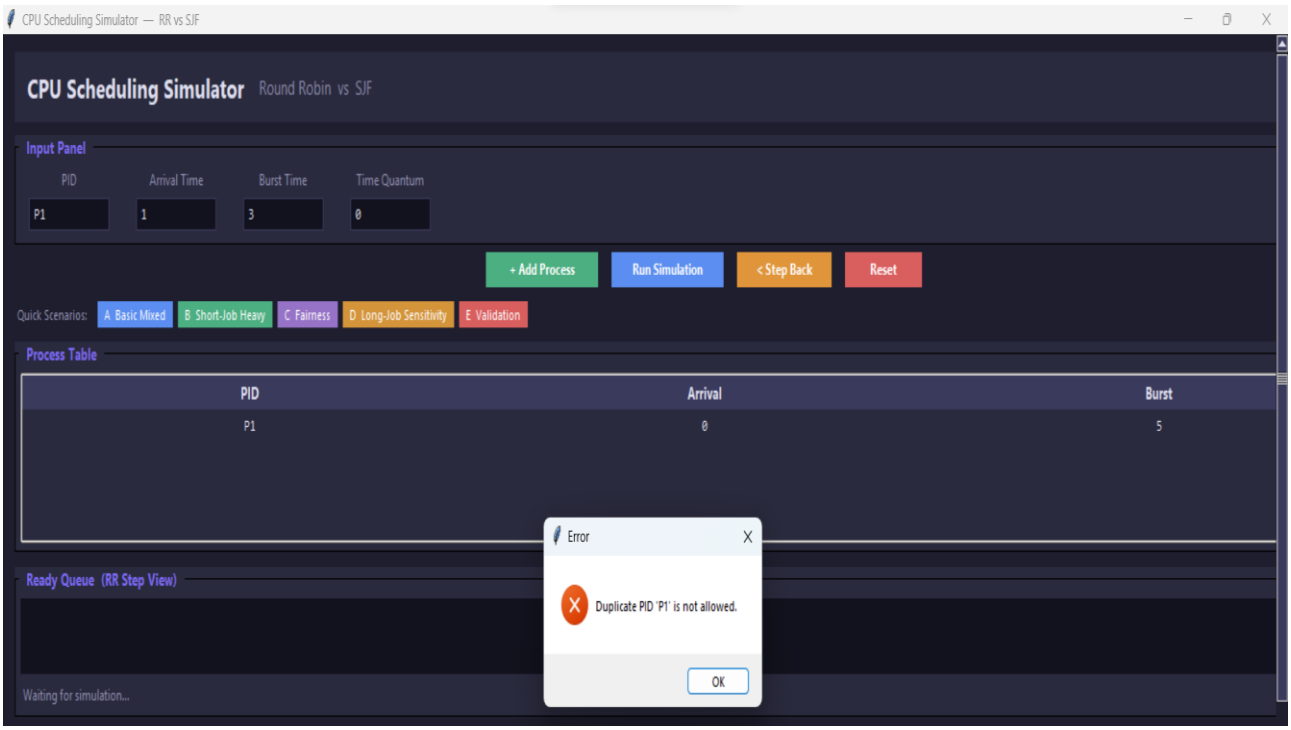


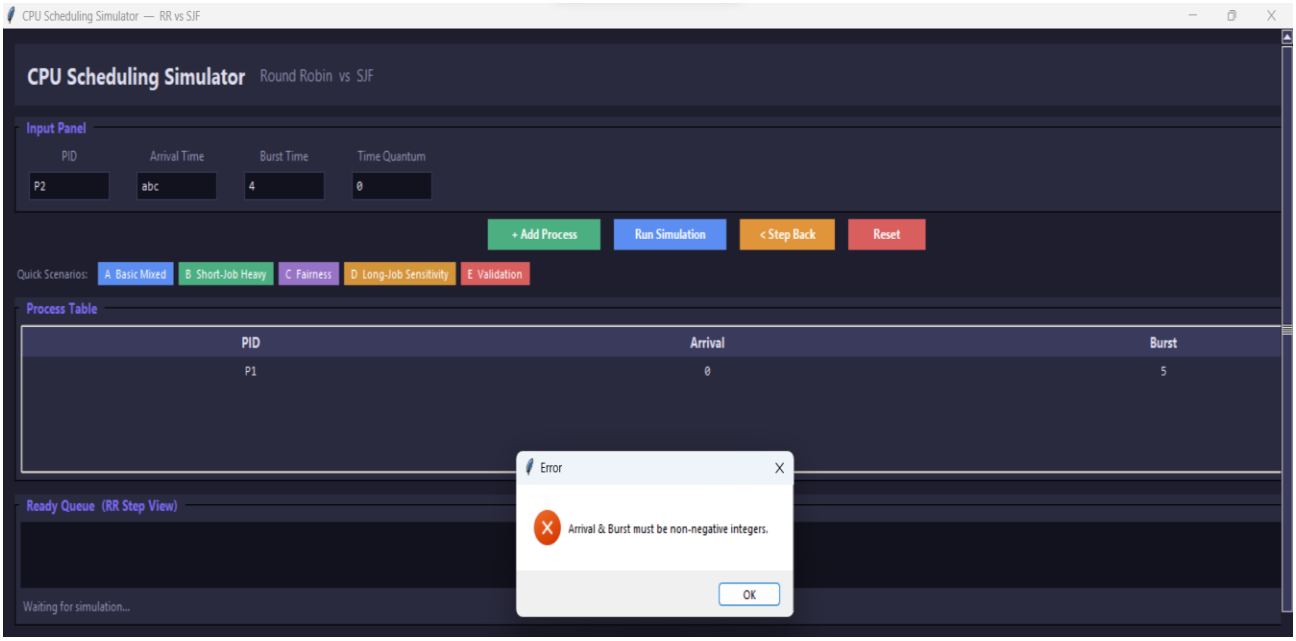


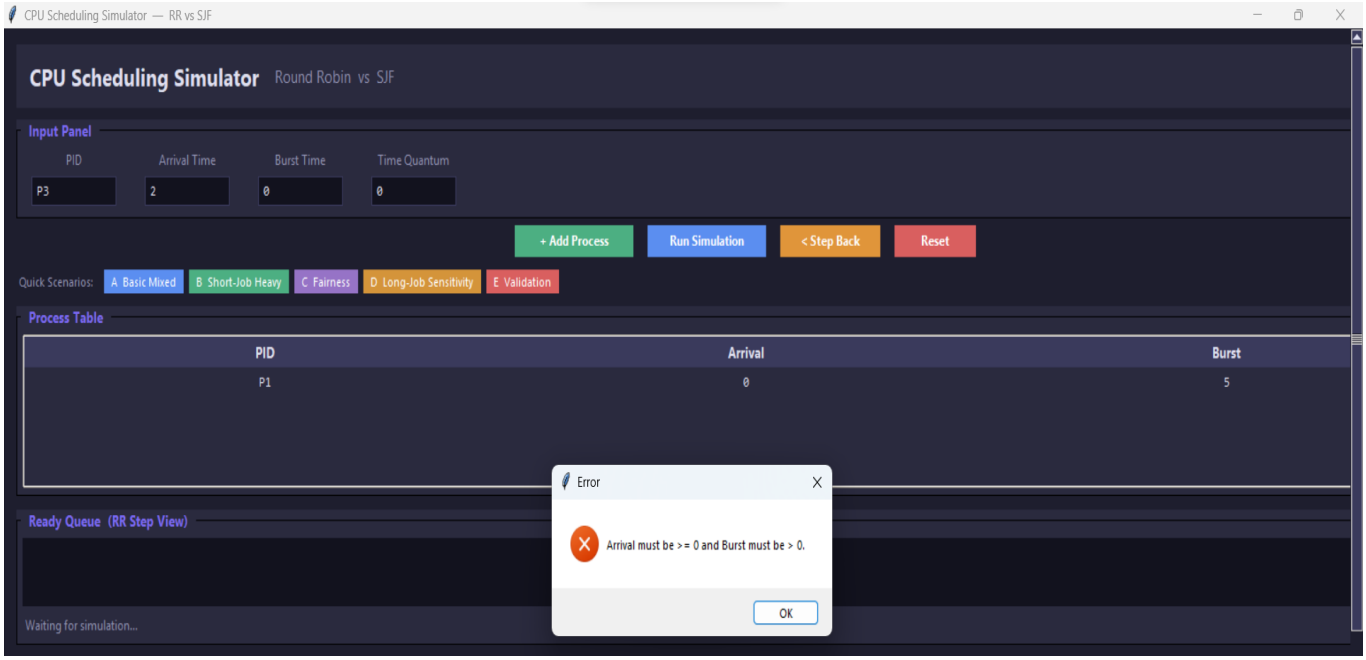
D. Long Job Sensitivity Case Screenshot Placeholder

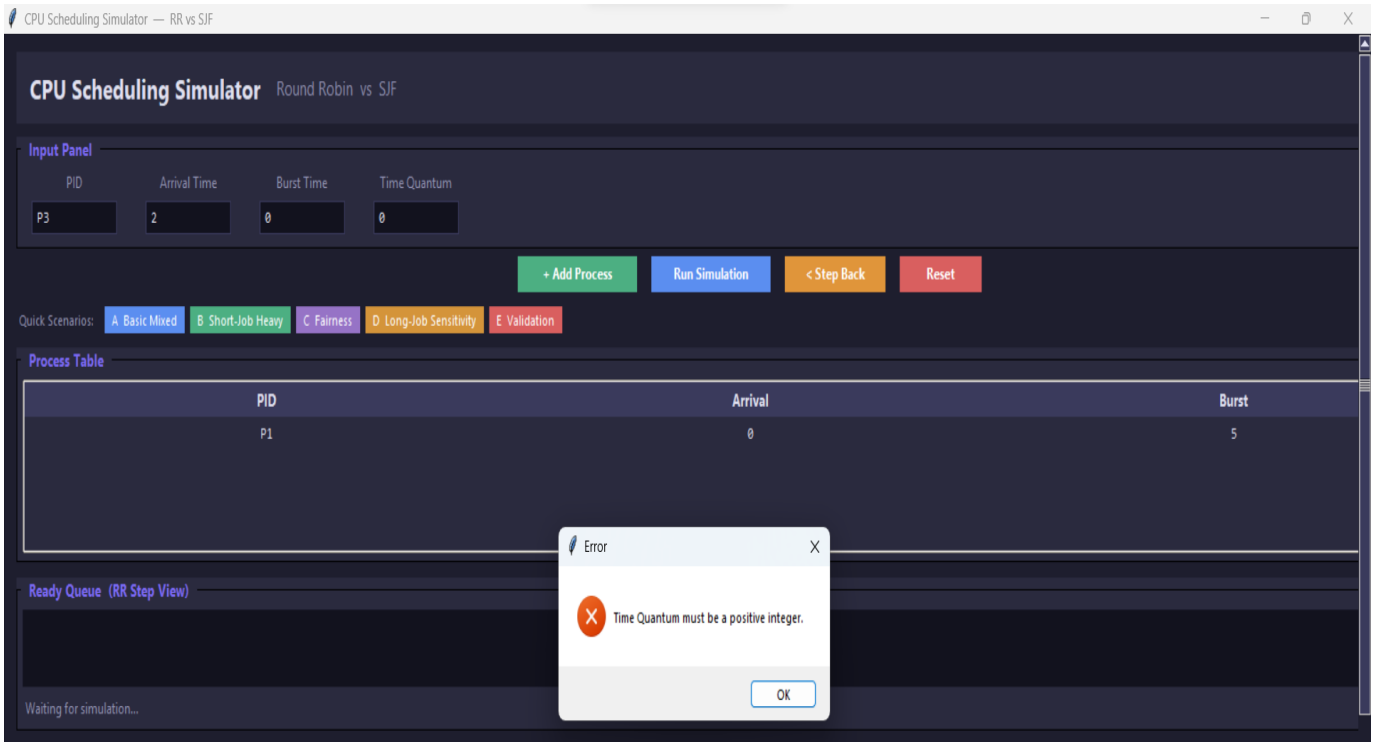


E. Validation Case Screenshot Placeholder






CPU Scheduling Simulator — RR vs SJF
CPU Scheduling Simulator
Round Robin vs SJF
Input Panel
PID
Arrival Time
Burst Time
Time Quantum
P3
2
0
0
+ Add Process
Run Simulation
< Step Back
Reset
Quick Scenarios:
A Basic Mixed
B Short-Job Heavy
C Fairness
D Long-Job Sensitivity
E Validation
Process Table
PID
Arrival
Burst
P1
0
5
Error
Arrival must be >= 0 and Burst must be > 0.
OK
Ready Queue (RR Step View)
Waiting for simulation...


CPU Scheduling Simulator — RR vs SJF
CPU Scheduling Simulator
Round Robin vs SJF
Input Panel
PID
Arrival Time
Burst Time
Time Quantum
P3
2
0
0
+ Add Process
Run Simulation
< Step Back
Reset
Quick Scenarios:
A Basic Mixed
B Short-Job Heavy
C Fairness
D Long-Job Sensitivity
E Validation
Process Table
PID
Arrival
Burst
P1
0
5
Error
Time Quantum must be a positive integer.
OK
Ready Queue (RR Step View)
Waiting for simulation...

Analysis Questions

1.Which algorithm gave lower average waiting time?

Shortest Job First (SJF) gave the lower average waiting time.
This is because SJF always selects the process with the shortest burst time first, which reduces the overall waiting time for shorter processes.

---

2. Which algorithm gave lower average response time?

Round Robin (RR) generally provided a better (lower) response time.
Since RR gives each process a time slice immediately, all processes start execution earlier compared to SJF.

---

3. Did Round Robin appear fairer across all processes?

Yes, **Round Robin appeared fairer**.
It distributes CPU time equally among all processes using time slicing, ensuring that no process is starved.

---

4. Did SJF complete short jobs more efficiently?

Yes, **SJF completed short jobs more efficiently**.
It prioritizes processes with the smallest burst time, allowing short jobs to finish very quickly.

---

5. How did the chosen quantum affect Round Robin behavior?

The **time quantum** significantly affected Round Robin performance:

- **Small quantum:** More fairness and better responsiveness, but higher context switching overhead.
- **Large quantum:** Fewer context switches and better efficiency, but RR starts behaving like FCFS and becomes less fair.

Overall, the selected quantum created a balance between responsiveness and system overhead.

6. Which algorithm would you recommend for the tested workload, and why?

- If the goal is **minimum waiting time and efficiency**, then **SJF is better**.
- If the goal is **fairness and responsiveness**, then **Round Robin is better**.

Recommendation:
For general workloads with mixed processes, **Round Robin is usually more practical**, while **SJF is better when short jobs dominate and burst times are known in advance**.

Conclusion

State which algorithm performed better on each major metric

- **Average Waiting Time:** SJF performed better
- **Response Time:** Round Robin performed better
- **Fairness:** Round Robin performed better
- **Short Job Completion:** SJF performed better

• State whether Round Robin appeared more balanced

Yes, **Round Robin appeared more balanced** because it distributes CPU time equally among all processes, preventing starvation and ensuring fairness.

• State whether SJF appeared more efficient

Yes, **SJF appeared more efficient**, especially in reducing average waiting time and completing short processes quickly.

• Explain the observed effect of the selected quantum

The **time quantum directly influenced Round Robin behavior**:

- A smaller quantum improved fairness and responsiveness but increased overhead due to frequent context switching.
- A larger quantum reduced overhead but decreased fairness and made the algorithm closer to First-Come First-Served (FCFS).

Therefore, the chosen quantum determined the trade-off between system efficiency and responsiveness.